

Full Git Commands Reference — Definitions, Syntax & Examples

A complete, categorized reference of Git commands: what each one does, its common flags, and a real-world usage example.

1. Setup & Configuration

`git config`

Definition: Sets or reads Git configuration values (user identity, editor, aliases, behavior) at system, global, or local (repo) level. **Syntax:** `git config [--system|--global|--local] <key> <value>` **Examples:**

```
bash

git config --global user.name "Priya Sharma"
git config --global user.email "priya@example.com"
git config --global core.editor "code --wait"
git config --list                # view all active settings
git config --global alias.co checkout # create a shortcut: 'git co' = 'git ch
```

`git init`

Definition: Creates a new, empty Git repository in the current folder (or a specified folder) by adding a hidden `.git` directory. **Syntax:** `git init [directory]` **Example:**

```
bash

mkdir my-project && cd my-project
git init                # initialize repo in current folder
git init my-app         # create 'my-app' folder and initialize it there
```

`git clone`

Definition: Downloads (copies) an existing repository — including its full history — from a remote source to your local machine. **Syntax:** `git clone <url> [folder-name]` **Example:**

```
bash

git clone https://github.com/torvalds/linux.git
git clone https://github.com/user/repo.git my-folder # clone into a custom
git clone --branch dev --single-branch <url>        # clone only one sp
git clone --depth 1 <url>                            # shallow clone
```

2. Basic Snapshotting

git status

Definition: Shows the current state of the working directory and staging area — which files are modified, staged, or untracked. **Syntax:** `git status [-s]` **Example:**

```
bash
git status          # detailed view
git status -s       # short view (M = modified, A = added, ?? = untracked)
```

git add

Definition: Moves changes from the working directory into the staging area (index), marking them to be included in the next commit. **Syntax:** `git add <file>...` **Example:**

```
bash
git add index.html          # stage one file
git add src/ tests/        # stage entire folders
git add .                  # stage everything in current directory
git add -A                 # stage everything in the whole repo, in
git add -p                 # interactively choose which changes (
```

git commit

Definition: Saves the currently staged changes as a new, permanent snapshot in the repository's history. **Syntax:** `git commit -m "<message>"` **Example:**

```
bash
git commit -m "Add user login form"
git commit -am "Fix typo in header"      # stage all tracked/modified files AND
git commit --amend -m "Corrected message" # replace the last commit's message
git commit --allow-empty -m "Trigger CI run" # commit with no file changes at
```

git diff

Definition: Shows line-by-line differences between the working directory, staging area, and commits. **Syntax:** `git diff [options] [commit1] [commit2]` **Example:**

```
bash
```

```
git diff                # unstaged changes (working dir vs staging area)
git diff --staged        # staged changes (staging area vs last commit)
git diff main feature    # compare two branches
git diff HEAD~2 HEAD     # compare two commits
```

git rm

Definition: Removes a file from the working directory and stages that removal for the next commit. **Syntax:** `git rm <file>` **Example:**

```
bash

git rm old-file.txt      # delete file and stage the deletion
git rm --cached secrets.env # stop tracking the file, but keep it on disk
git rm -r build/         # recursively remove a whole folder
```

git mv

Definition: Renames or moves a file and automatically stages the change. **Syntax:** `git mv <old-path> <new-path>` **Example:**

```
bash

git mv index.js main.js
```

3. Viewing History

git log

Definition: Displays the commit history of the repository, newest first by default. **Syntax:**

`git log [options]` **Example:**

```
bash

git log                # full detailed history
git log --oneline      # compact: one line per commit
git log --oneline --graph --all --decorate # visual branch graph with labels
git log -p -2          # show diffs for the last 2 commits
git log --author="Priya" # filter by author
git log --since="2 weeks ago" # filter by date
git log --grep="fix"     # search commit messages
git log -- path/to/file.js # history of one specific file
```

git show

```
bash
```

```
git show a1b2c3d          # show full details of a specific commit
git show HEAD~1           # show the commit before the current one
git show v1.0.0           # show details of an annotated tag
```

git blame

Definition: Shows who last modified each line of a file, and in which commit — useful for tracking down when/why a bug was introduced. **Syntax:** `git blame <file>` **Example:**

```
bash
```

```
git blame utils.py
git blame -L 10,25 utils.py  # restrict to lines 10-25
```

git shortlog

Definition: Summarizes commit history grouped by author — useful for contributor statistics. **Syntax:** `git shortlog -sn` **Example:**

```
bash
```

```
git shortlog -sn          # commit counts per author, sorted highest first
```

4. Branching & Merging

git branch

Definition: Lists, creates, renames, or deletes branches. **Syntax:** `git branch [options] [branch-name]` **Example:**

```
bash
```

```
git branch                # list local branches
git branch -a             # list all branches (local + remote)
git branch new-feature    # create a branch (doesn't switch to it)
git branch -d old-feature # delete a branch (safe — must be merged)
git branch -D old-feature # force-delete, even if unmerged
git branch -m old-name new-name # rename a branch
```

git switch

Definition: Switches between existing branches, or creates and switches to a new one (modern replacement for part of `git checkout`). **Syntax:** `git switch <branch>` **Example:**

```
bash

git switch main           # switch to the 'main' branch
git switch -c feature/cart # create AND switch to a new branch
git switch -              # switch back to the previously checked-out
```

git checkout

Definition: The classic, multi-purpose command to switch branches, restore files, or check out a specific commit. **Syntax:** `git checkout <branch|commit|file>` **Example:**

```
bash

git checkout feature-branch # switch branches (older syntax)
git checkout -b new-branch  # create + switch in one step
git checkout -- file.txt    # discard uncommitted changes to a file
git checkout a1b2c3d        # check out a specific commit (details)
```

git merge

Definition: Combines the changes from one branch into the currently checked-out branch. **Syntax:** `git merge <branch>` **Example:**

```
bash

git checkout main
git merge feature-branch # merge feature-branch into main
git merge --no-ff feature-branch # force a merge commit even if fast-forward
git merge --squash feature-branch # combine all commits from feature-branch
git merge --abort           # cancel a merge that has conflicts
```

git rebase

Definition: Replays your branch's commits on top of another branch's latest commit, producing a linear history instead of a merge commit. **Syntax:** `git rebase <branch>` **Example:**

```
bash
```

```
git checkout feature
git rebase main          # replay feature's commits on top of main
git rebase -i HEAD~3     # interactive rebase: squash/reorder/edit th
git rebase --continue    # resume after resolving a conflict
git rebase --abort       # cancel and return to pre-rebase stat
```

`git cherry-pick`

Definition: Applies the changes from one specific commit (from any branch) onto your current branch. **Syntax:** `git cherry-pick <commit-hash>` **Example:**

```
bash

git cherry-pick a1b2c3d          # apply one specific commit
git cherry-pick a1b2c3d b4e5f6g # apply multiple commits
git cherry-pick --continue      # after resolving a conflict duri
```

5. Undoing Changes

`git restore`

Definition: Restores files in the working directory or unstages files (modern replacement for part of `git checkout`). **Syntax:** `git restore <file>` **Example:**

```
bash

git restore file.txt          # discard uncommitted changes to a file
git restore --staged file.txt # unstage a file (keep the edits)
git restore .                 # discard ALL uncommitted changes
```

`git reset`

Definition: Moves the current branch pointer to a different commit, optionally changing the staging area and working directory too. **Syntax:** `git reset [--soft|--mixed|--hard] <commit>` **Example:**

```
bash

git reset --soft HEAD~1      # undo last commit, keep changes staged
git reset --mixed HEAD~1     # undo last commit, unstage changes (default mode)
git reset --hard HEAD~1      # undo last commit AND discard all changes (des
git reset --hard origin/main # force local branch to match the remote exa
```

git revert

Definition: Creates a new commit that undoes the changes made by a previous commit — safe for shared/pushed history. **Syntax:** `git revert <commit>` **Example:**

```
bash

git revert a1b2c3d          # revert one specific commit
git revert HEAD             # revert the most recent commit
git revert --no-commit a1b2c3d  # stage the revert without committing yet
```

git reflog

Definition: Shows a log of everywhere `HEAD` has pointed — a safety net to recover "lost" commits after resets or rebases. **Syntax:** `git reflog` **Example:**

```
bash

git reflog                # view HEAD's movement history
git reset --hard HEAD@{2}  # jump back to a previous state shown in r
```

git clean

Definition: Removes untracked files from the working directory. **Syntax:** `git clean [-n] [-f] [-d]` **Example:**

```
bash

git clean -n      # dry run - preview what would be deleted
git clean -f      # actually delete untracked files
git clean -fd     # also delete untracked directories
```

6. Stashing

git stash

Definition: Temporarily shelves (saves) uncommitted changes so you can switch context, then reapply them later. **Syntax:** `git stash [subcommand]` **Example:**

```
bash
```

```
git stash                                # stash tracked changes
git stash -u                            # also stash untracked new files
git stash save "WIP: navbar"            # stash with a descriptive label
git stash list                          # show all saved stashes
git stash show -p stash@{0}             # view the diff inside a stash
git stash apply                         # reapply latest stash (keep
git stash pop                           # reapply latest stash AND
git stash drop stash@{1}                # delete one specific s
git stash clear                         # delete all stashes
```

7. Working with Remotes

git remote

Definition: Manages the set of remote repositories (bookmarks pointing to other copies of the project) tracked by your local repo. **Syntax:** `git remote [subcommand]` **Example:**

```
bash

git remote -v                          # list remotes with their URLs
git remote add origin git@github.com:user/repo.git # register a remote named
git remote remove origin                # unregister a remote
git remote rename origin upstream       # rename a remote
git remote set-url origin <new-url>     # change a remote's UR
```

git fetch

Definition: Downloads new commits/branches from a remote, without merging them into your local branches. **Syntax:** `git fetch <remote>` **Example:**

```
bash

git fetch origin                        # fetch updates from 'origin'
git fetch --all                         # fetch from every registered remote
```

git pull

Definition: Downloads new commits from a remote AND immediately merges (or rebases) them into your current branch. **Syntax:** `git pull <remote> <branch>` **Example:**

```
bash

git pull origin main                   # fetch + merge
git pull --rebase originmain           # fetch + rebase (keeps linear history)
```

git push

Definition: Uploads your local commits to a remote repository. **Syntax:** `git push <remote> <branch>` **Example:**

```
bash

git push origin main           # push commits to 'main' on 'origin'
git push -u origin main        # push AND set upstream tracking
git push origin --delete old-branch # delete a branch on the remote
git push --tags                 # push all local tags
git push --force-with-lease     # safer force-push (checks)
```

8. Tagging

git tag

Definition: Marks a specific commit as significant (usually a release point) with a readable name. **Syntax:** `git tag [-a] <tag-name> [-m "message"]` **Example:**

```
bash

git tag v1.0.0                 # lightweight tag on current commit
git tag -a v1.0.0 -m "First release" # annotated tag (recommended — has
git tag -a v0.9.0 a1b2c3d      # tag a specific past commit
git tag -d v1.0.0              # delete a local tag
git push origin v1.0.0         # push one tag to the rem
git push origin --tags         # push all tags
```

9. Inspection & Internals

git cat-file

Definition: Displays the type or content of any object in Git's internal database (blob, tree, commit, tag). **Syntax:** `git cat-file [-t|-p] <hash>` **Example:**

```
bash

git cat-file -t a1b2c3d        # show object type
git cat-file -p a1b2c3d        # pretty-print object content
```

git ls-tree

Definition: Lists the contents of a tree object (i.e., a directory listing at a specific commit).

Syntax: `git ls-tree <commit>` **Example:**

```
bash

git ls-tree HEAD          # top-level files/folders in the current commit
git ls-tree -r HEAD       # recursively list every file
```

git hash-object

Definition: Computes (and optionally stores) the SHA-1 hash Git would use for a file's content. **Syntax:** `git hash-object [-w] <file>` **Example:**

```
bash

git hash-object file.txt    # just compute the hash
git hash-object -w file.txt # compute AND store it as a blob object
```

git fsck

Definition: Checks the integrity and connectivity of objects in the repository's database.

Syntax: `git fsck` **Example:**

```
bash

git fsck --full
```

git gc

Definition: Cleans up the repository — compresses loose objects into packfiles and removes unreachable data to save space. **Syntax:** `git gc` **Example:**

```
bash

git gc          # standard cleanup
git gc --aggressive # more thorough (slower) optimization
```

10. Searching & Debugging

git bisect

Definition: Uses binary search across commit history to find the exact commit that introduced a bug. **Syntax:** `git bisect start` **Example:**

```
bash
```

```
git bisect start
git bisect bad                # current commit is broken
git bisect good v1.2.0        # this earlier commit was fine
# Git checks out a midpoint – test it, then mark it:
git bisect good              # or
git bisect bad
git bisect reset              # exit bisect mode when done
git bisect run npm test       # automate the whole process with a test
```

git grep

Definition: Searches for a text pattern across all tracked files in the repository. **Syntax:** `git grep "<pattern>"` **Example:**

```
bash

git grep "TODO"                # find every "TODO" comment in the codebase
git grep -n "function login"   # show line numbers too
```

11. Advanced / Power-User Commands

git worktree

Definition: Allows multiple branches to be checked out into separate folders simultaneously, sharing one `.git` database. **Syntax:** `git worktree add <path> <branch>`

Example:

```
bash

git worktree add ../hotfix hotfix-branch    # check out 'hotfix-branch' in a separate folder
git worktree list                            # list all active worktrees
git worktree remove ../hotfix                # clean up when done
```

git submodule

Definition: Embeds another Git repository inside your repository as a linked reference (pointer + commit SHA). **Syntax:** `git submodule add <url> <path>` **Example:**

```
bash

git submodule add https://github.com/user/lib.git libs/lib
git submodule update --init --recursive    # pull in submodule content after cloning
git submodule update --remote               # update submodules to their latest commit
```

git archive

Definition: Creates a compressed archive (zip/tar) of files from a specific commit or branch.

Syntax: `git archive --format=zip <branch> -o <output.zip>` **Example:**

```
bash
git archive --format=zip HEAD -o project.zip
```

git filter-repo / git filter-branch

Definition: Rewrites repository history — commonly used to permanently remove a file (like an accidentally committed secret) from all past commits. **Syntax:** `git filter-repo --path <file> --invert-paths` **Example:**

```
bash
git filter-repo --path secrets.env --invert-paths # remove secrets.env from
```

git rerere

Definition: Records how you resolved a merge conflict so Git can automatically reapply that same resolution if the identical conflict happens again. **Syntax:** `git config --global rerere.enabled true` **Example:**

```
bash
git config --global rerere.enabled true
```

git bundle

Definition: Packages repository data (commits, branches) into a single file that can be transferred without a network connection. **Syntax:** `git bundle create <file> <branch>`

Example:

```
bash
git bundle create backup.bundle main # bundle 'main' branch into a portab
git clone backup.bundle restored-repo # restore a repo from a bundle
```

git sparse-checkout

Definition: Checks out only specific subfolders of a large repository instead of the entire tree — useful for huge monorepos. **Syntax:** `git sparse-checkout set <folder>` **Example:**

```
bash
```

```
git sparse-checkout init --cone
git sparse-checkout set frontend/ shared/
```

12. Quick Command Lookup Table

Command	One-line Purpose
<code>git init</code>	Start a new repository
<code>git clone</code>	Copy an existing repository
<code>git status</code>	Show current changes state
<code>git add</code>	Stage changes
<code>git commit</code>	Save a snapshot
<code>git log</code>	View commit history
<code>git diff</code>	Compare changes
<code>git branch</code>	Manage branches
<code>git switch</code> / <code>checkout</code>	Change current branch
<code>git merge</code>	Combine branches
<code>git rebase</code>	Replay commits on a new base
<code>git cherry-pick</code>	Apply one commit elsewhere
<code>git reset</code>	Move branch pointer / undo commits
<code>git revert</code>	Safely undo via a new commit
<code>git reflog</code>	Recover lost history
<code>git stash</code>	Temporarily shelve changes
<code>git remote</code>	Manage remote links
<code>git fetch</code>	Download remote changes (no merge)
<code>git pull</code>	Download + merge remote changes
<code>git push</code>	Upload local commits

Command	One-line Purpose
<code>git tag</code>	Mark a release point
<code>git blame</code>	See who changed each line
<code>git bisect</code>	Binary-search for a bad commit
<code>git gc</code>	Clean up/optimize the repo
<code>git worktree</code>	Multiple branches checked out at once
<code>git submodule</code>	Embed another repo inside yours

Every command above works completely offline except `clone`, `fetch`, `pull`, and `push`, which require a remote connection.